CASE STUDY 2

Smart Road Network

using Kruskal's Algorithm

Minimum Spanning Tree for HMDA Urban Connectivity

Department	FED
Subject	DSA-2
Academic Year	I-III Semester (2025-26) : { 1st year, 3rd term }
Faculty Name	Dr. Jayaram Boga
Student Name	P. Bindhu Madhavi
Roll Number	2520030077
Branch	CSE

Course Outcome CO2:
Apply Kruskal's Algorithm and Union-Find to construct Minimum Spanning Trees, demonstrating cycle detection, greedy edge selection, and O(E log E) performance in real-world road network and connectivity scenarios.

SCENARIO

The Hyderabad Metropolitan Development Authority (HMDA) is developing a smart road network to improve connectivity between major urban areas: Ameerpet (A), Banjara Hills (B), Charminar (C), Dilsukhnagar (D), Gachibowli (G), and Kukatpally (K), all connected through a central hub at Hitech City (H).

Each possible road has a construction cost (Rs. crore). The objective is to design a **minimum-cost network** connecting all locations, while maintaining redundancy between H and G (IT hub requirement).

Candidate Roads and Costs

Edge	Cost (Rs. Cr)	Edge	Cost (Rs. Cr)	Edge	Cost (Rs. Cr)
D-G	2	H-A	6	G-K	7
B-C	3	C-D	6	H-C	8
A-B	4	H-D	7	H-B	9
H-G	5	H-K	10	A-K	11

Redundancy Requirement: At least two edge-disjoint paths must exist between Hitech City (H) and Gachibowli (G).

WORK DONE SUMMARY

In this case study, the Minimum Spanning Tree (MST) for the HMDA road network was constructed using Kruskal's algorithm. All 12 candidate edges were sorted in non-decreasing order of weight, and the algorithm was simulated step by step.

For each edge, cycle detection was applied using connected components — 6 edges were accepted and 6 were rejected. The final MST cost is **Rs. 27 crore**.

The algorithm was also implemented in Java using a Union-Find (DSU) data structure. Finally, the MST was evaluated for the H-G redundancy requirement and the optimal additional edge (C-D, Rs. 6 crore) was identified, giving a total augmented network cost of **Rs. 33 crore**.

IMPLEMENTATION DETAILS

Detail	Description
1. Edge Sorting	All 12 edges sorted non-decreasing: D-G(2), B-C(3), A-B(4), H-G(5), H-A(6), C-D(6), H-D(7), G-K(7), H-C(8), H-B(9), H-K(10), A-K(11).
2. Kruskal's Greedy Selection	At each step, the minimum-weight edge is examined. If endpoints are in different components it is accepted and merged; otherwise rejected to prevent a cycle.
3. Cycle Detection via Components	Initial: 7 isolated components {A}{B}{C}{D}{G}{H}{K}. Each accepted edge reduces the component count by 1. MST is complete when 1 component remains.
4. Union-Find (DSU)	find(x) traverses the parent chain to return the root. If find(u)==find(v), the edge creates a cycle — reject. union(u,v) merges two components.
5. Java Implementation	Edges sorted using Comparable. parent[] array initialised with parent[i]=i. union() returns false if both endpoints share the same root. Returns Rs. 27 crore.
6. MST Result	Selected edges: D-G(2), B-C(3), A-B(4), H-G(5), H-A(6), G-K(7). Topology: C-B-A-H-G-K with D connected to G. Total cost = Rs. 27 crore.
7. Redundancy Analysis	MST has only one path H-G. Adding C-D (Rs. 6 Cr) creates second path H-A-B-C-D-G. Augmented network cost = Rs. 33 crore.

QUESTIONS

(a) Construct the minimum spanning tree using Kruskal's algorithm. Arrange all edges in non-decreasing order and simulate the selection process. Clearly indicate for each edge whether it is accepted or rejected, justify using cycle detection, and show how connected components evolve during each step.

(b) Design and implement Kruskal's algorithm using a union-find data structure. Include find() and union() operations ensuring no cycles are formed. Explain how sorted edges are processed and how total cost is calculated.

(c) Evaluate whether the constructed MST satisfies the redundancy requirement between Hitech City (H) and Gachibowli (G). If not, identify the minimum-cost edge to add for an alternative path and justify your choice.

ANSWERS

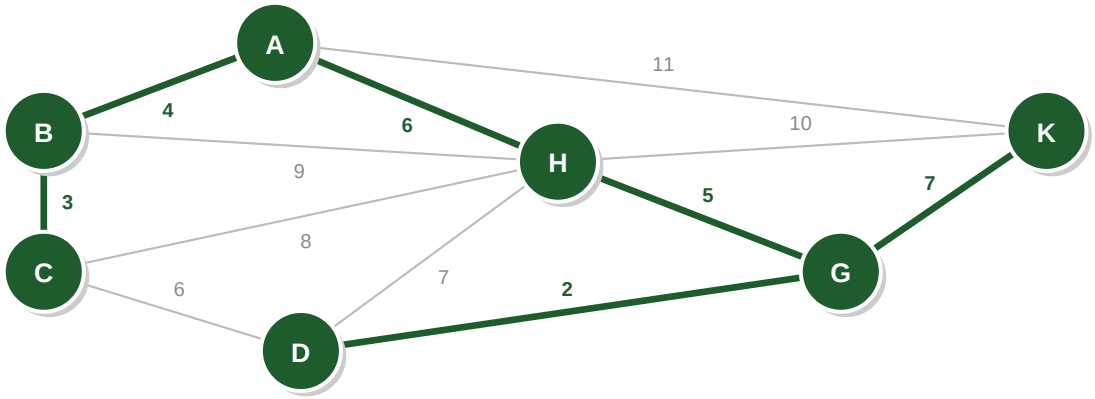
(a) MST Using Kruskal's Algorithm

Initial components: {A} {B} {C} {D} {G} {H} {K} — 7 isolated nodes

Step	Edge	Cost	Decision	Reason	Components After
1	D-G	2	Accept	No cycle	{DG} {A} {B} {C} {H} {K}
2	B-C	3	Accept	No cycle	{BC} {DG} {A} {H} {K}
3	A-B	4	Accept	No cycle	{ABC} {DG} {H} {K}
4	H-G	5	Accept	No cycle	{HDG} {ABC} {K}
5	H-A	6	Accept	Connects 2 components	{ABCDGH} {K}
6	C-D	6	Reject	C & D already connected	No change
7	H-D	7	Reject	H & D already connected	No change
8	G-K	7	Accept	K joins MST	{ABCDGHK} — complete
9	H-C	8	Reject	Cycle formed	No change
10	H-B	9	Reject	Cycle formed	No change
11	H-K	10	Reject	Cycle formed	No change
12	A-K	11	Reject	Cycle formed	No change

Selected Edges: D-G(2) + B-C(3) + A-B(4) + H-G(5) + H-A(6) + G-K(7) = **Rs. 27 crore**

MST Network — Green edges selected, Grey edges rejected



(b) Kruskal's Algorithm — Java with Union-Find

find(x) returns root representative. If find(u)==find(v) then cycle → reject. union(x,y) merges two components after accepting an edge.

```
import java.util.*;
class Kruskal {
static class Edge implements Comparable<Edge> {
int u, v, w;
Edge(int u,int v,int w){ this.u=u; this.v=v; this.w=w; }
public int compareTo(Edge e){ return this.w - e.w; }
}

static int find(int[] parent, int x) {
while (parent[x] != x) x = parent[x];
return x;
}

static boolean union(int[] parent, int x, int y) {
int px = find(parent, x), py = find(parent, y);
if (px == py) return false; // same root = cycle
parent[px] = py;
return true;
}

static int kruskal(int n, List<Edge> edges) {
Collections.sort(edges);
int[] parent = new int[n];
for (int i=0; i<n; i++) parent[i] = i;
int totalCost=0, edgeCount=0;
for (Edge e : edges) {
if (union(parent, e.u, e.v)) {
totalCost += e.w;
if (++edgeCount == n-1) break;
}
}
return totalCost; // returns 27
}
}
```

Output: kruskal() returns **Rs. 27 crore** | Edges accepted: D-G, B-C, A-B, H-G, H-A, G-K

(c) Redundancy Requirement Analysis

The MST provides exactly **one path** between H and G (direct H-G edge). If this edge fails, G is completely disconnected from the network. A second edge-disjoint path must be added.

Option	Edge Added	Cost	New Path via Added Edge	Verdict
1 (Optimal)	C-D	Rs. 6 Cr	H-A-B-C-D-G	**BEST**
2	H-D	Rs. 7 Cr	H-D-G	Higher cost
3	H-K	Rs. 10 Cr	H-K-G	Highest cost

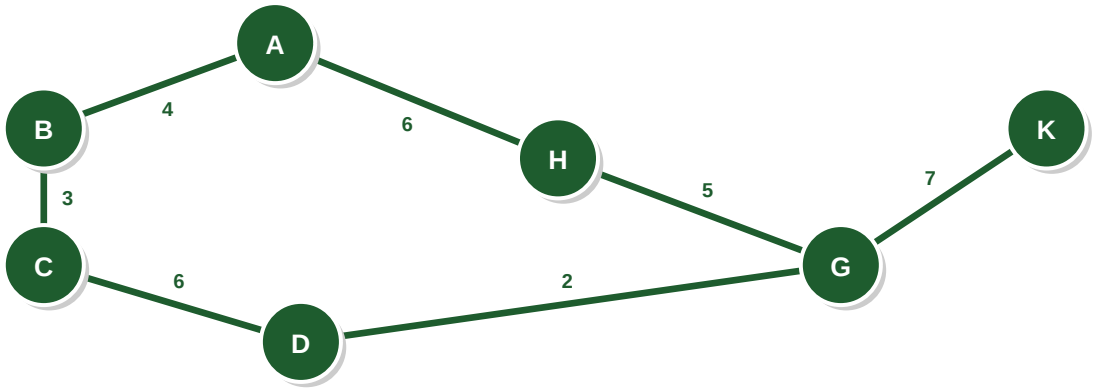
Optimal choice: Add edge C-D (Rs. 6 crore).

Path 1 (MST): H → G (direct).

Path 2 (new): H → A → B → C → D → G.

Augmented network cost = Rs. 27 + Rs. 6 = Rs. 33 crore.

Augmented Network — MST + C-D redundancy edge (Total: Rs. 33 Cr)



RESULT OUTPUT SUMMARY

Edge	D-G	B-C	A-B	H-G	H-A	G-K	TOTAL
Cost (Rs. Cr)	2	3	4	5	6	7	27

Part	Result
(a) — MST Result	Total MST Cost = Rs. 27 crore. 6 edges accepted, 6 rejected. Topology: C-B-A-H-G-K with D connected to G via D-G edge.
(b) — Union-Find Output	kruskal() returns Rs. 27 crore. 12 edges processed, 6 accepted (n-1=6), 6 rejected due to cycle detection.
(c) — Augmented Network	MST alone does NOT satisfy the H-G redundancy requirement. Add edge C-D (Rs. 6 crore) for a second disjoint path. Final augmented cost = Rs. 33 crore.

Algorithm Complexity Summary

Operation	Time Complexity	Space Complexity
Sort Edges	O(E log E)	O(E)
find() (path compression)	O(alpha(V)) = O(1)	O(V)
union() by rank	O(alpha(V)) = O(1)	O(V)
Overall Kruskal's	O(E log E)	O(V + E)

GitHub Link: https://github.com/bindhu-java/DSA2-CO-WISE-/tree/main

Student Signature	Faculty Signature
_____	_____